Week 6 — Develop First Modeling Approach

Next-Month Domestic Airline Passenger-Demand Forecasting

===== WEEK 6 START =====

Assignment: One method, three hyperparameter variations, training and validation metrics, choose a winner **only using validation data**.

Proposed first model: Ridge regression on the two Week 5 PCA predictors (`PC1` , `PC2`).

- This is a simple linear model ahead of potentially more complex methods in later weeks.
- We use the exact Week 3 chronological target and split logic.
- We do not evaluate the held-out test partition in Week 6.

The transformed input pipeline was fitted on the training period only. Predicted passenger counts are clipped at 0 for practical validity, consistently across all candidate models.

1. Import the Week 6 reusable source

Only one pipeline implements the calculations. This notebook calls `Src/week6_src_ridge_models.py` ; no duplicate model logic is pasted into the notebook.

In [1]:
```
from pathlib import Path
import importlib.util
import sys
from IPython.display import display, Image

def project_root(start=None):
    start = (start or Path.cwd()).resolve()
    for folder in [start, *start.parents]:
        if all((folder / name).is_dir() for name in ('Src', 'Data', 'Notebooks')):
            return folder
    raise FileNotFoundError('Repository root not found.')

ROOT = project_root()
SRC = ROOT / 'Src' / 'week6_src_ridge_models.py'
spec = importlib.util.spec_from_file_location('week6_ridge', SRC)
week6 = importlib.util.module_from_spec(spec)
sys.modules[spec.name] = week6
spec.loader.exec_module(week6)
REPORTS = ROOT / 'Reports' / 'week6'
TABLES = REPORTS / 'week6_tables'
FIGURES = REPORTS / 'week6_figures'
print('Root:', ROOT)
print('Source:', SRC)
print('Settings:', week6.MODEL_SETTINGS)
```

```
Root: C:\
Source: C:\Src\week6_src_ridge_mod els.py
Settings: [('Ridge_A', 1.0), ('Ridge_B', 10000.0), ('Ridge_C', 1000000.0)]
```

2. Reconstruct Week 5 PCA inputs and Week 3 split

Reusing the Week 5 `Src/week5_src_features_dimensionality.py` creates the complete PC1/PC2 matrices from the **official BTS source archives**.

- The Week 5 PCA medians, scaler, and PCA components are fitted to training only.
- Previous Week 5 *sample CSVs* won't be used for model evaluation because they are only small previews.

In [2]:
```
partitions, split_summary, input_source = week6._load_week5_data(ROOT)
print('Input source:', input_source)
display(split_summary)
print('Training rows:', len(partitions['Training']))
print('Validation rows:', len(partitions['Validation']))
assert len(partitions['Training']) > len(partitions['Validation']) > 0
assert partitions['Training']['ForecastMonth'].max() < partitions['Validation']['ForecastMonth'].min()
print('Training–validation chronology verified. Test outcomes not evaluated.')
```

```
Input source: Week 5 source (reconstructed full PCA from official BTS ZIPs)
```

	Split	Rows	ForecastMonths	FirstReferenceMonth	LastReferenceMonth	FirstForecastMonth	LastForecastMonth	RowPercent	Mo
0	Training	1062603	95	2015-01-01	2022-11-01	2015-02-01	2022-12-01	67.842078	
1	Validation	239395	20	2022-12-01	2024-07-01	2023-01-01	2024-08-01	15.284216	
2	Test	264291	21	2024-08-01	2026-04-01	2024-09-01	2026-05-01	16.873706	

```
Training rows: 1062603
Validation rows: 239395
Training-validation chronology verified. Test outcomes not evaluated.
```

3. Define three Ridge regularization strengths and evaluation measures

Ridge minimizes squared training errors with an additional coefficient penalty controlled by `alpha`. Stronger alpha shrinks coefficients more and may reduce overfitting at the cost of underfitting. The settings are deliberately spread across orders of magnitude, given the large training sample.

Candidate	Alpha	Regularization level
Ridge_A	1	Weak
Ridge_B	10,000	Moderate
Ridge_C	1,000,000	Strong

MAE (passengers) is the primary selection metric because typical absolute deviation is easy for capacity planners to interpret. **RMSE** (passengers) penalizes large misses more heavily, and **R^2** summarizes fit relative to predicting the mean. Metrics are calculated separately on **training and validation**. There is no hyperparameter search on the test data.

In [3]:
```
results = week6.train_and_compare(
    partitions['Training'],
    partitions['Validation'],
    split_summary,
    data_source=input_source,
)
display(results.metrics_table.round(3))
assert results.metrics_long.shape[0] == 6
assert results.metrics_table.shape[0] == 3
assert set(results.metrics_long['Split']) == {'Training', 'Validation'}
```

	Model	Alpha	Training_MAE	Training_RMSE	Training_R2	Validation_MAE	Validation_RMSE	Validation_R2	MAE_Gap_ValidationMinu
0	Ridge_A	1.0	3331.959	5787.133	0.444	3257.177	5887.098	0.473	
1	Ridge_B	10000.0	3332.425	5789.185	0.443	3259.035	5890.276	0.473	
2	Ridge_C	1000000.0	3395.138	5985.650	0.405	3418.394	6163.435	0.422	

4. Compare training-versus-validation errors

The difference between training and validation errors helps assess whether changing alpha improves generalization, though temporal drift may also contribute. A smaller training error is not sufficient grounds to select the winner.

In [4]:
```
display(results.comparison.round(3))
print('Selected by lowest VALIDATION MAE:', results.winner, 'alpha:', results.winner_alpha)
```

	Model	Alpha	Validation_MAE	Validation_RMSE	Validation_R2	MAE_Gap_ValidationMinusTraining
0	Ridge_A	1.0	3257.177	5887.098	0.473	-74.782
1	Ridge_B	10000.0	3259.035	5890.276	0.473	-73.390
2	Ridge_C	1000000.0	3418.394	6163.435	0.422	23.256

```
Selected by lowest VALIDATION MAE: Ridge_A alpha: 1.0
```

5. Save reproducible Week 6 tables, figures, and winning model

The winner is selected based on validation performance only. The selected estimator is saved in `Models/week6/`, with an associated JSON manifest describing its inputs and winning metrics. The saved estimator requires the matching Week 5 PCA transformation for future inference;

it is not a stand-alone raw-data predictor.

In [5]:
```
week6.save_outputs(results, ROOT)
for path in sorted(TABLES.glob('*')):
    print('TABLE:', path.name)
for path in sorted(FIGURES.glob('*.png')):
    print('FIGURE:', path.name)
print('MODEL:', ROOT / 'Models' / 'week6' / 'week6_selected_ridge.joblib')
```

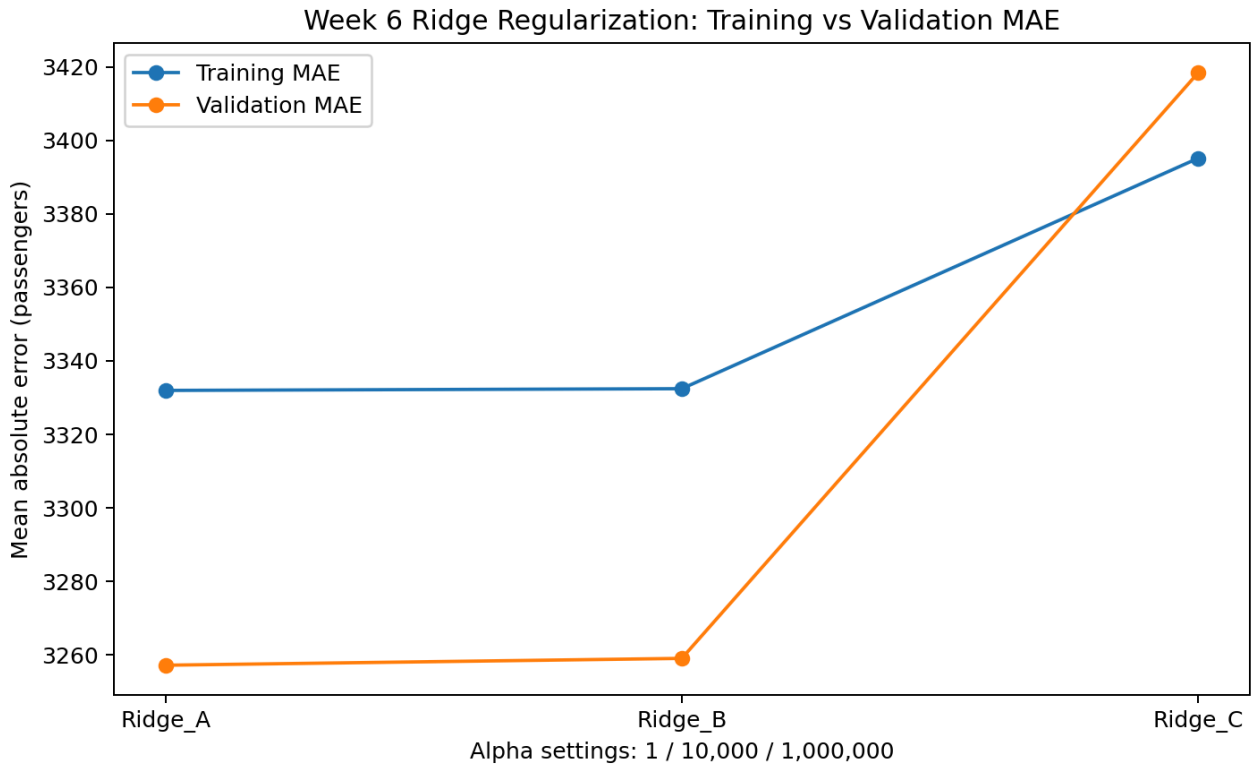
```
TABLE: week6_metrics_train_validation_long.csv
TABLE: week6_models_metrics_comparison.csv
TABLE: week6_split_summary.csv
TABLE: week6_validation_ranking.csv
TABLE: week6_winner_summary.json
FIGURE: week6_figure_mae_by_alpha.png
FIGURE: week6_figure_rmse_by_alpha.png
FIGURE: week6_figure_validation_actual_vs_predicted.png
MODEL: C:\Models\week6\week6_selec ted_ridge.joblib
```

6. Inspect comparison and diagnostic figures

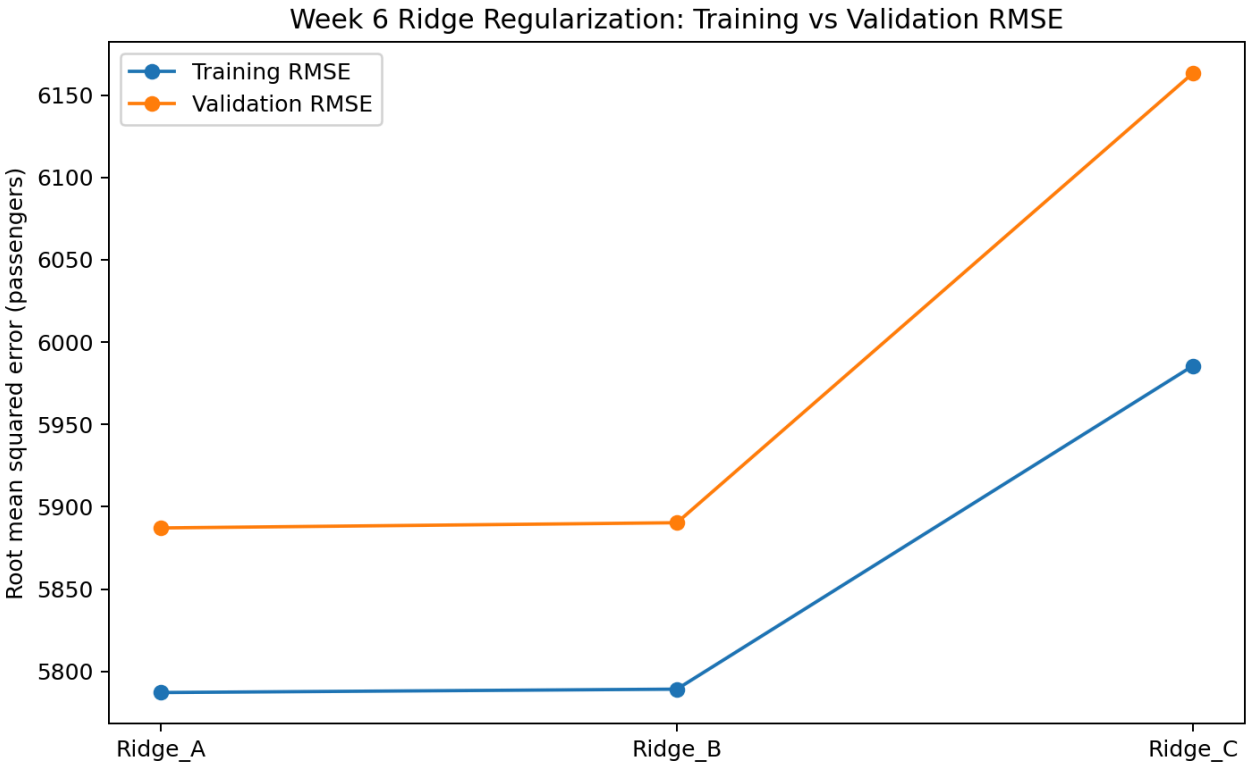
Validation actual-versus-predicted is sampled only for visualization. The MAE/RMSE/R^2 metrics above are computed using **all** training and validation rows.

In [6]:
```
for filename in [
    'week6_figure_mae_by_alpha.png',
    'week6_figure_rmse_by_alpha.png',
    'week6_figure_validation_actual_vs_predicted.png',
]:
    path = FIGURES / filename
    print(filename)
    display(Image(filename=str(path)))
```

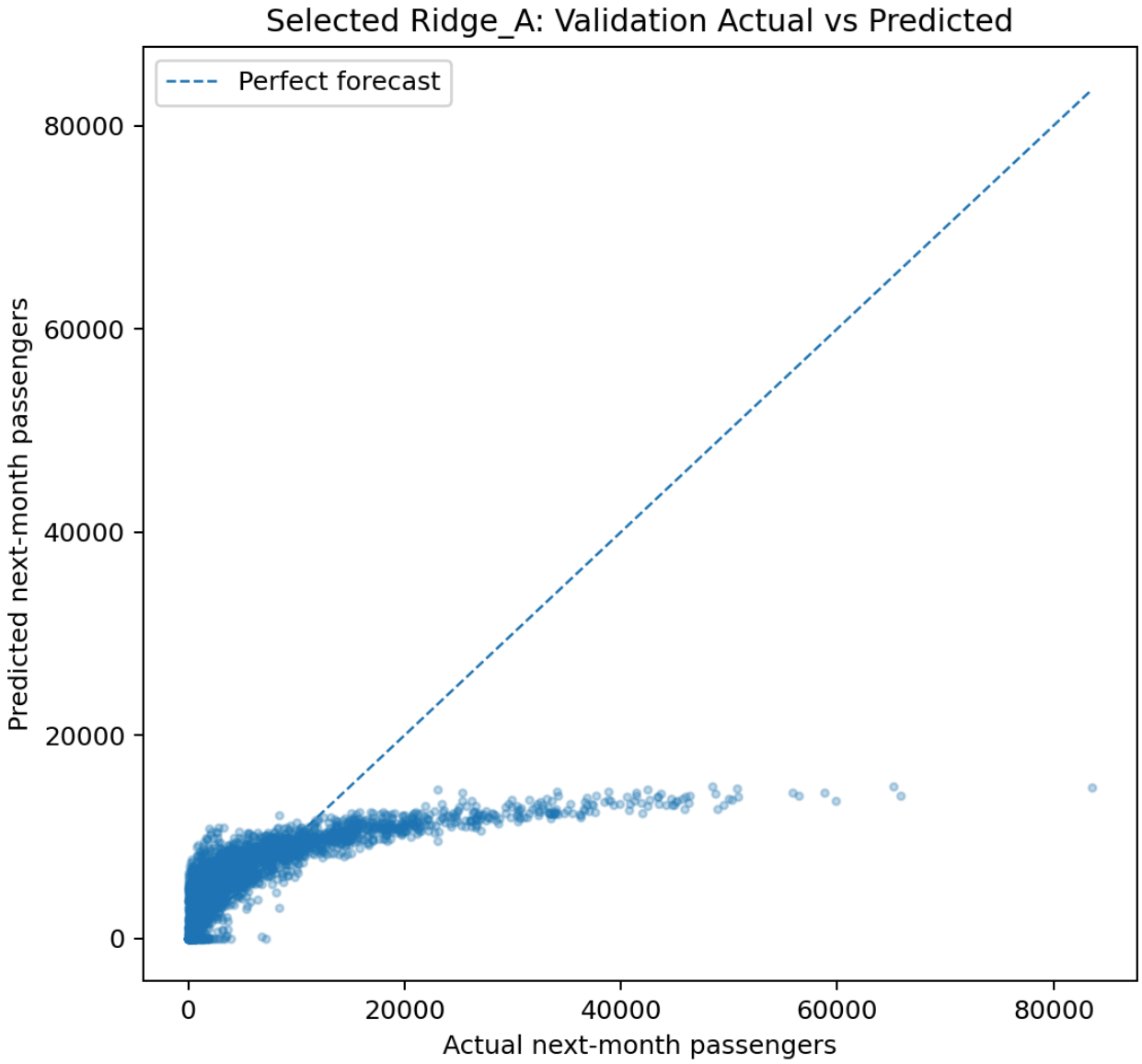
```
week6_figure_mae_by_alpha.png
```




```
week6_figure_rmse_by_alpha.png
```



week6_figure_validation_actual_vs_predicted.png



7. Conclusion and final integrity checks

The Week 6 winner is a *provisional validation-selected model*.

- Generalization performance remains unknown until a later, explicitly authorized final test-set evaluation.
- The winning Ridge model on two PCA dimensions provides an interpretable benchmark in terms of model complexity, though the PCA components themselves are less transparent than original carrier/route variables.

The model does **not** receive future target values as predictors, and the Week 6 test partition is excluded from tuning and evaluation.

In [7]:
```
from pathlib import Path
import numpy as np

assert len(results.metrics_table) == 3
assert (results.metrics_table['Training_MAE'] >= 0).all()
assert (results.metrics_table['Validation_MAE'] >= 0).all()
assert (results.metrics_table['Training_RMSE'] >= 0).all()
assert (results.metrics_table['Validation_RMSE'] >= 0).all()
assert (np.isfinite(results.metrics_table[['Training_MAE','Validation_MAE','Training_RMSE','Validation_RMSE','Training_R2','Va
assert (TABLES / 'week6_winner_summary.json').exists()
assert (FIGURES / 'week6_figure_mae_by_alpha.png').exists()
assert (ROOT / 'Models' / 'week6' / 'week6_selected_ridge.joblib').exists()
print('All Week 6 checks passed.')
print('===== WEEK 6 END =====')
```

```
All Week 6 checks passed.
===== WEEK 6 END =====
```